

End-to-End Automotive Cybersecurity Assessment – Mein White-Box Security Ansatz

Fahrzeugsysteme werden immer stärker vernetzt – und sind daher zunehmend Cyberrisiken ausgesetzt. Ich habe dieses Konzept zur End-to-End-Sicherheitsvalidierung entwickelt, um meinen Ansatz bei der Bewertung der Cybersicherheit im Automobilbereich zu veranschaulichen: von der Bedrohungsmodellierung und White-Box-Analyse bis hin zu CAN-/UDS-Tests, Firmware-Prüfung, Automatisierung und Risikobewertung. Mein Schwerpunkt: die Ermittlung der Grundursachen, die Entwicklung reproduzierbarer Sicherheitstests und die Umsetzung technischer Erkenntnisse in konkrete Verbesserungsmaßnahmen.

Phase 1 – Scope Definition: Die Grundlage jedes Automotive Security Assessments

Bevor technische Tests beginnen, muss zunächst der Sicherheitsumfang klar definiert werden. Ein effektives Automotive Security Assessment startet nicht mit dem ersten Angriff, sondern damit, die zu schützenden Systeme und die tatsächlichen Angriffsflächen zu verstehen.

Zunächst werden alle sicherheitsrelevanten Komponenten des Fahrzeugs identifiziert. Dazu gehören die internen Kommunikationsnetzwerke CAN, CAN FD, Automotive Ethernet und LIN sowie die zentralen Steuergeräte (ECUs), die sicherheitskritische Funktionen übernehmen. Je nach Fahrzeugarchitektur kommt außerdem FlexRay hinzu.

Im Fokus stehen unter anderem:

-  Gateway ECU als zentrale Kommunikationsinstanz
-  Telematics Control Unit (TCU) für Mobilfunk- und OTA-Kommunikation
-  Infotainment-Systeme mit externen Konnektivitätsdiensten
-  Body Control Module (BCM)
-  Powertrain-Steuergeräte
-  ADAS-Controller für Fahrerassistenzsysteme

Ebenso werden alle extern erreichbaren Schnittstellen betrachtet, über die ein potenzieller Angreifer mit dem Fahrzeug interagieren könnte:

- OBD-II Diagnoseschnittstelle
- USB-Ports
- Bluetooth
- WLAN
- Mobilfunkverbindungen
- Over-the-Air-(OTA)-Update-Mechanismen

Im nächsten Schritt wird der eigentliche Testumfang definiert. Für jede Komponente werden konkrete Sicherheitsziele festgelegt. So wird beispielsweise die Gateway ECU auf Kommunikationskontrolle, die UDS-Diagnose auf Authentifizierungsmechanismen (Security Access), die Firmware mittels Reverse Engineering und der Bootloader hinsichtlich der Secure-Boot-Implementierung untersucht. Zusätzlich werden die Fahrzeugnetzwerke durch gezielte CAN-Manipulationen und Fuzzing-Tests auf ihre Widerstandsfähigkeit geprüft.

Ein sauber definierter Scope bildet die Grundlage für ein reproduzierbares und nachvollziehbares Security Assessment. Erst wenn die Architektur, die Kommunikationspfade und die Angriffsflächen vollständig verstanden sind, können Sicherheitsmechanismen systematisch validiert und reale Risiken zuverlässig bewertet werden.

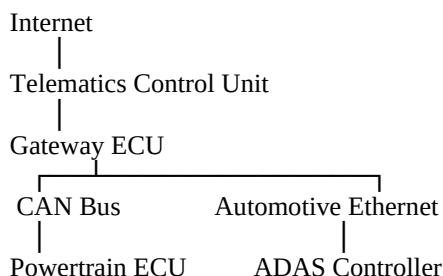
Phase 2 – Systemarchitektur verstehen: Sicherheit beginnt mit Systemverständnis

Nachdem der Testumfang definiert wurde, folgt der wichtigste Schritt eines White-Box-Security-Assessments: das vollständige Verständnis der Fahrzeugarchitektur.

Bevor Schwachstellen identifiziert werden können, muss zunächst nachvollzogen werden, wie Steuergeräte miteinander kommunizieren, welche Daten ausgetauscht werden und welche Komponenten sicherheitskritische Entscheidungen treffen. Denn viele Sicherheitsprobleme entstehen nicht durch einzelne Programmierfehler, sondern durch das Zusammenspiel mehrerer Systeme.

Im Rahmen der Architekturanalyse wird daher zunächst ein vollständiges Modell des Fahrzeugs erstellt. Dazu gehören die Netzwerktopologie, die Kommunikationspfade zwischen den einzelnen ECUs sowie die logischen Vertrauensgrenzen (*Trust Boundaries*) innerhalb der Fahrzeugarchitektur.

Ein typisches Beispiel ist die Kommunikation zwischen einer Telematics Control Unit (TCU), die mit Mobilfunk- oder Cloud-Diensten verbunden ist, und dem internen Fahrzeugnetzwerk. Häufig erfolgt der Datenfluss über eine Gateway-ECU, die Nachrichten zwischen unterschiedlichen Bussystemen wie CAN und Automotive Ethernet weiterleitet.



Aus der Perspektive eines Security Engineers stehen dabei nicht nur die Architektur selbst, sondern vor allem die sicherheitsrelevanten Fragestellungen im Mittelpunkt.

- Welche Steuergeräte besitzen privilegierte Rechte innerhalb des Fahrzeugnetzwerks?
- Wo befinden sich sicherheitskritische Vertrauensgrenzen zwischen internen und externen Kommunikationsbereichen?
- Welche Kommunikationskanäle sind authentifiziert oder kryptografisch geschützt?
- Können Nachrichten zwischen unterschiedlichen Netzwerken unzureichend gefiltert oder manipuliert werden?
- Welche Komponenten stellen im Falle einer Kompromittierung ein Risiko für sicherheitskritische Fahrzeugfunktionen dar?

Mithilfe dieser Analyse entsteht ein vollständiges Verständnis der internen Kommunikationsstrukturen. Nur so lassen sich potenzielle Angriffspfade systematisch identifizieren und realistische Angriffsszenarien entwickeln.

Dieses Architekturverständnis bildet die Grundlage für alle nachfolgenden Sicherheitsanalysen im Rahmen eines White-Box-Penetrationstests – vom Threat Modeling über Kommunikationsanalysen bis hin zum Firmware-Review. Gleichzeitig unterstützt diese Phase die Root-Cause-Analyse, da Sicherheitsprobleme nicht isoliert, sondern im Kontext der gesamten Fahrzeugarchitektur betrachtet und bewertet werden können.

Ein erfolgreicher Penetrationstest beginnt nicht mit einem Exploit, sondern mit dem Verständnis der Architektur. Wer die Kommunikationswege, Vertrauensgrenzen und Abhängigkeiten eines Fahrzeugs kennt, kann nicht nur Schwachstellen, sondern auch deren Ursachen identifizieren.

Phase 3 – Threat Modeling: Angriffe verstehen, bevor sie stattfinden

Nachdem die Fahrzeugarchitektur vollständig analysiert wurde, folgt der nächste entscheidende Schritt eines Automotive Security Assessments: das Threat Modeling. Das Ziel dieser Phase besteht nicht darin, unmittelbar Schwachstellen zu finden, sondern vielmehr darin, die existierenden Angriffsmöglichkeiten sowie die besonders schützenswerten Komponenten eines Fahrzeugs systematisch zu analysieren. Moderne Fahrzeuge bestehen aus zahlreichen vernetzten Steuergeräten, externen Kommunikationsschnittstellen und komplexen Softwarekomponenten. Nicht jede Verbindung stellt automatisch ein Sicherheitsrisiko dar. Erst durch strukturiertes Threat Modeling lassen sich potenzielle Angriffswege identifizieren, priorisieren und gezielt untersuchen.

Für die Analyse werden etablierte Methoden eingesetzt, die sich in der Automotive Cybersecurity bewährt haben:

- **STRIDE** zur systematischen Identifikation von Bedrohungen wie Spoofing, Tampering oder Denial of Service.
- **Attack Trees**, um mögliche Angriffspfade bis zum eigentlichen Schutzziel nachvollziehbar darzustellen.
- **Threat Analysis and Risk Assessment (TARA)** gemäß **ISO/SAE 21434**, um Risiken hinsichtlich Eintrittswahrscheinlichkeit und möglicher Auswirkungen zu bewerten.

Ein realistisches Angriffsszenario wäre beispielsweise die Manipulation von CAN-Nachrichten. Dabei verschafft sich ein Angreifer über eine externe Schnittstelle Zugriff auf das Fahrzeugnetzwerk und injiziert manipulierte CAN-Frames. Wenn diese Kommunikation sicherheitskritische Steuergeräte erreicht, besteht die Möglichkeit, Fahrzeugfunktionen unberechtigt zu beeinflussen.

Schützenswertes Asset: Bremssteuergerät (Brake ECU)

Bedrohung: Unautorisierte Steuerung sicherheitskritischer Funktionen (*Unauthorized Control*)

Mögliche Auswirkungen

- Manipulation von Steuerbefehlen
- Beeinträchtigung der Fahrzeugsicherheit
- Verlust der Integrität sicherheitskritischer Kommunikation

Risikobewertung ● **High** – Aufgrund der möglichen Auswirkungen auf die funktionale Sicherheit besitzt dieses Szenario höchste Priorität.

Im Rahmen des Threat Modelings werden anschließend weitere potenzielle Angriffsszenarien identifiziert und bewertet.

Bedrohung	Angriffsszenario	Mögliche Auswirkung
CAN Injection	Manipulation von CAN-Frames	Beeinflussung von Fahrzeugfunktionen
Firmware Manipulation	Unsicherer Bootloader oder fehlende Signaturprüfung	Ausführung manipulierter Firmware
Diagnosemissbrauch	Unzureichend geschützter UDS-Zugriff	Unautorisierte ECU-Funktionen
OTA-Angriff	Manipulation eines Software-Updates	Kompromittierung mehrerer Steuergeräte

Das eigentliche Ziel dieser Phase besteht jedoch nicht darin, möglichst viele Bedrohungen aufzulisten. Viel wichtiger ist es, die wahrscheinlichsten und kritischsten Angriffspfade zu identifizieren und daraus konkrete Testfälle für die folgenden Phasen des Security Assessments abzuleiten. Threat Modeling bildet somit eine Brücke zwischen Architekturverständnis und technischer Sicherheitsprüfung. Anstatt zufällig einzelne Schwachstellen zu suchen, werden White-Box-Analysen, Kommunikationsanalysen und Penetrationstests gezielt auf die Bereiche mit dem größten Risiko für das Gesamtsystem fokussiert.

Beim Threat Modeling wird die Perspektive eines Angreifers eingenommen – lange bevor der erste technische Test beginnt. Wer versteht, welche Systeme besonders wertvoll sind und wie sie angegriffen werden könnten, kann Sicherheitsmaßnahmen gezielt entwickeln, testen und kontinuierlich verbessern.

Phase 4 – White-Box Analysis: Sicherheit beginnt im Quellcode



Nachdem die Architektur und potenzielle Angriffsszenarien analysiert wurden, richtet sich der Fokus nun auf die Implementierung. Denn viele kritische Sicherheitslücken entstehen nicht durch fehlende Sicherheitskonzepte, sondern durch Fehler im Code, unsichere Konfigurationen oder unzureichend abgesicherte Softwarekomponenten. Anders als bei einem klassischen Black-Box-Penetrationstest ermöglicht der White-Box-Ansatz einen vollständigen Einblick in die Software eines Steuergeräts. Neben der Systemarchitektur stehen nun auch der Quellcode, die Implementierungsdetails und die Sicherheitsmechanismen zur Verfügung. So lassen sich Schwachstellen identifizieren, bevor sie sich im Fahrzeugverhalten bemerkbar machen. Der Fokus liegt dabei nicht auf der Suche nach einzelnen Programmierfehlern, sondern auf der Bewertung der Sicherheitsarchitektur der Software als Ganzes. Das Ziel besteht darin, Sicherheitslücken bereits auf Implementierungsebene zu identifizieren und ihre Ursache zu verstehen.

Während eines Source-Code-Reviews werden unter anderem folgende typische Fragestellungen behandelt:

- Werden sensible Informationen wie Passwörter, Schlüssel oder Tokens direkt im Quellcode gespeichert?
- Werden kryptografische Verfahren korrekt implementiert oder kommen veraltete bzw. unsichere Algorithmen zum Einsatz?
- Existieren Speicherfehler wie Buffer Overflows, Integer Overflows oder Use-after-Free-Schwachstellen?
- Werden sicherheitskritische Fehler sauber behandelt oder entstehen dadurch neue Angriffsflächen?

Ein klassisches Beispiel hierfür ist der unsichere Umgang mit Speicheroperationen in der Programmiersprache C. Neben der manuellen Codeanalyse kommen in einem professionellen Security Assessment verschiedene Static Application Security Testing (SAST)-Werkzeuge zum Einsatz. Diese unterstützen dabei, potenzielle Schwachstellen automatisiert zu identifizieren und die Codequalität systematisch zu bewerten. Gerade in Embedded-Systemen entstehen viele Sicherheitsprobleme nicht durch einen einzelnen Fehler, sondern durch das Zusammenspiel aus Speicherverwaltung, Kommunikationsschnittstellen, Kryptografie und sicherheitskritischer Logik. Aus diesem Grund betrachtet die White-Box-Analyse nicht nur einzelne Codezeilen, sondern die gesamte Implementierung im Kontext der Fahrzeugarchitektur.

Das Ziel besteht darin, die Ursachen einer Schwachstelle zu verstehen und Verbesserungen abzuleiten. Ein Code-Review ist somit zentral für die Root-Cause-Analyse und die Softwareentwicklung im Automotive-Bereich.

Phase 5 – Kommunikationsanalyse: Wenn Fahrzeugnetzwerke zur Angriffsfläche werden

Nachdem die Architektur, das Bedrohungsmodell und die Implementierung analysiert wurden, folgt die Untersuchung der Kommunikation zwischen den Steuergeräten. In modernen Fahrzeugen werden sicherheitskritische Entscheidungen innerhalb von Millisekunden über verschiedene Bussysteme ausgetauscht. Genau deshalb ist die Analyse der Fahrzeugkommunikation ein zentraler Bestandteil eines White-Box-Security-Assessments.

Im Mittelpunkt stehen dabei insbesondere CAN und Automotive Ethernet. Diese beiden Technologien übernehmen heute in nahezu jedem vernetzten Fahrzeug eine Schlüsselrolle. Zwar ist CAN seit Jahrzehnten als zuverlässiges Kommunikationssystem etabliert, jedoch wurde es ursprünglich nicht mit einem Fokus auf Cybersecurity entwickelt. Nachrichten werden standardmäßig weder verschlüsselt noch authentifiziert. Prinzipiell kann jedes Steuergerät, das Zugriff auf den Bus erhält, Nachrichten senden, empfangen oder manipulieren. Aus Sicht eines Angreifers entsteht dadurch eine potenzielle Angriffsfläche, die systematisch untersucht werden muss.

Analyse des CAN-Bus

Zunächst muss der Datenverkehr innerhalb des Fahrzeugnetzwerks vollständig verstanden werden. Hierzu werden alle CAN-Nachrichten aufgezeichnet, dekodiert und in Bezug auf ihr Kommunikationsverhalten analysiert.

Dabei stehen unter anderem die folgenden Fragestellungen im Fokus:

- Welche Steuergeräte kommunizieren miteinander?
- Welche Nachrichten werden zyklisch übertragen?
- Welche CAN-Identifizier steuern sicherheitskritische Funktionen?
- Existieren ungewöhnliche oder nicht dokumentierte Kommunikationsmuster?
- Werden Nachrichten durch das Gateway ausreichend gefiltert?

Für diese Analysen werden etablierte Werkzeuge aus den Bereichen Automotive-Entwicklung und Security-Testing eingesetzt:

- **CANoe** zur Simulation und Analyse komplexer Fahrzeugnetzwerke
- **Wireshark** zur Untersuchung verschiedener Kommunikationsprotokolle
- **SocketCAN** als Linux-basierte CAN-Schnittstelle
- **python-can** zur Entwicklung automatisierter Testskripte

Ein einfaches Beispiel zur Analyse eingehender CAN-Nachrichten mit Python könnte wie folgt aussehen:

```
import can
bus = can.interface.Bus (
    channel="can0",
    interface="socketcan" )
for msg in bus:
    print(msg.arbitration_id, msg.data)
```

Der Mehrwert liegt jedoch nicht im Auslesen einzelner Frames, sondern in der systematischen Auswertung der Kommunikation. Nur so lassen sich Kommunikationsmuster erkennen, Abhängigkeiten zwischen Steuergeräten nachvollziehen und potenzielle Angriffspfade identifizieren.

CAN Fuzzing – Robustheit unter realistischen Angriffsbedingungen

Auf die passive Analyse folgt die aktive Sicherheitsprüfung. Beim CAN-Fuzzing werden gezielt manipulierte oder unerwartete Nachrichten in das Fahrzeugnetzwerk eingespeist, um das Verhalten einzelner Steuergeräte unter ungewöhnlichen Bedingungen zu untersuchen. Dabei wird beobachtet, wie das jeweilige Steuergerät auf fehlerhafte oder unerwartete Eingaben reagiert.

Unter anderem stehen folgende Fragestellungen im Mittelpunkt:

- Führt die Nachricht zu einem unerwarteten ECU-Reset?
- Werden Eingabedaten ausreichend validiert?
- Entstehen Fehlfunktionen oder undefinierte Systemzustände?
- Bleibt die Kommunikation stabil oder beeinflusst der Test andere Steuergeräte?

Gerade diese Tests liefern oft wertvolle Erkenntnisse für die Ursachenanalyse, da sie Schwachstellen aufdecken, die im normalen Fahrzeugbetrieb niemals zutage treten würden.

Analyse von Automotive Ethernet

Mit der zunehmenden Digitalisierung übernehmen leistungsfähige Steuergeräte immer häufiger die Kommunikation über Automotive Ethernet. Dabei erzeugen Infotainment-Systeme, ADAS-Plattformen oder OTA-Updates neue Anforderungen an die Performance, gleichzeitig entstehen jedoch auch zusätzliche Angriffsflächen.

Im Rahmen des Assessments wird deshalb untersucht.:

- welche Netzwerkdienste erreichbar sind,
- welche Protokolle verwendet werden,
- ob unnötige Services aktiviert sind,
- ob verschlüsselte Verbindungen korrekt implementiert wurden,
- und ob Kommunikationsschnittstellen den aktuellen Sicherheitsanforderungen entsprechen.

Hierzu kommen unter anderem folgende Werkzeuge zum Einsatz:

- **Nmap** zur Identifikation erreichbarer Netzwerkdienste
- **Wireshark** zur Analyse von Ethernet-Kommunikation
- **Scapy** zur Erstellung und Manipulation individueller Netzwerkpakete sowie zur Entwicklung reproduzierbarer Security-Tests

Die Ergebnisse dieser Analysen liefern nicht nur Hinweise auf potenzielle Schwachstellen, sondern ermöglichen auch eine Bewertung der Sicherheitsarchitektur des gesamten Fahrzeugnetzwerks.

Kommunikationsanalysen bilden somit die Brücke zwischen theoretischem Threat Modeling und der praktischen Durchführung von Penetrationstests. Sie zeigen, ob Sicherheitsmechanismen unter realen Bedingungen tatsächlich wirksam sind und ob es Kommunikationswege gibt, über die sich Angriffe auf sicherheitskritische Steuergeräte durchführen lassen.

Fahrzeugnetzwerke sind das Nervensystem moderner Automobile. Wer ihre Kommunikationsstrukturen versteht, erkennt nicht nur einzelne Schwachstellen, sondern kann auch nachvollziehen, wie sich ein Angriff durch das gesamte System ausbreiten könnte. Genau hier beginnt effektive Automotive Cybersecurity.

Phase 6 – Diagnose-Sicherheit (UDS): Wenn Wartungsfunktionen zur Angriffsfläche werden

Moderne Fahrzeuge sind schon lange mehr als nur mechanische Systeme. Sie bestehen aus einer Vielzahl vernetzter Steuergeräte, deren Konfiguration, Wartung und Aktualisierung über standardisierte Diagnoseprotokolle erfolgt. Eines der wichtigsten Protokolle ist dabei Unified Diagnostic Services (UDS) gemäß ISO 14229. UDS wurde entwickelt, um Werkstätten und Herstellern umfangreiche Diagnose- und Programmierfunktionen bereitzustellen. Gleichzeitig eröffnet dieser Funktionsumfang potenzielle Angriffsflächen, wenn Sicherheitsmechanismen unzureichend implementiert oder falsch konfiguriert sind.

Deshalb ist die Sicherheitsbewertung der Diagnosefunktionen ein zentraler Bestandteil eines White-Box-Security-Assessments.

Diagnose verstehen, bevor sie getestet wird

In einem ersten Schritt wird analysiert, welche Diagnosefunktionen von den einzelnen Steuergeräten unterstützt werden und welche Berechtigungen für deren Nutzung notwendig sind.

Der Fokus liegt dabei insbesondere auf jenen UDS-Services, die direkten Einfluss auf den Zustand oder die Software einer ECU haben.

UDS Service	Funktion
0x10	Wechsel der Diagnosesitzung (<i>Diagnostic Session Control</i>)
0x11	Neustart eines Steuergeräts (<i>ECU Reset</i>)
0x22	Auslesen interner Fahrzeug- und Diagnosedaten (<i>Read Data by Identifier</i>)
0x27	Zugriff auf geschützte Funktionen (<i>Security Access</i>)
0x34	Start eines Firmware-Downloads (<i>Request Download</i>)
0x36	Übertragung neuer Firmware-Daten (<i>Transfer Data</i>)

Diese Dienste sind für die Entwicklung und Wartung unverzichtbar. Falls sie jedoch in die Hände von Angreifern gelangen, können sie zur Manipulation von Steuergeräten oder sogar zur Kompromittierung sicherheitskritischer Fahrzeugfunktionen missbraucht werden.

Security Access – Der wichtigste Schutzmechanismus

Ein besonderes Augenmerk gilt dem Security Access Service (0x27). Er schützt privilegierte Diagnosefunktionen vor unautorisiertem Zugriff und stellt oft die letzte Sicherheitsbarriere vor Programmier- oder Konfigurationsfunktionen einer ECU dar. Typischerweise basiert dieser Mechanismus auf einem Seed-Key-Verfahren.

Der Ablauf ist einfach:

1. Die ECU erzeugt einen zufälligen Seed.
2. Der Diagnosetester berechnet mithilfe eines geheimen Algorithmus daraus den passenden Schlüssel (Key).
3. Nur wenn der berechnete Schlüssel korrekt ist, werden die geschützte Funktionen freigegeben.

Ein typischer Kommunikationsablauf sieht beispielsweise wie folgt aus:

Tester → ECU

Request:

27 01

ECU → Tester

Response:

67 01 AA BB CC

Die Antwort enthält den vom Steuergerät generierten Seed, der anschließend zur Berechnung des gültigen Schlüssels verwendet wird. Aus Sicht eines Security Engineers beginnt an dieser Stelle jedoch erst die eigentliche Analyse.

Sicherheitsmechanismen gezielt validieren

Im Rahmen des Assessments wird geprüft, ob die Implementierung des Security-Access-Mechanismus den aktuellen Sicherheitsanforderungen entspricht.

Dabei stehen unter anderem die folgenden Fragestellungen im Mittelpunkt:

- Ist der verwendete Seed ausreichend zufällig oder lässt er sich vorhersagen?
- Werden moderne kryptografische Verfahren zur Schlüsselberechnung eingesetzt?
- Existieren Schutzmechanismen gegen Brute-Force-Angriffe?
- Werden nach mehreren Fehlversuchen Wartezeiten (Delay Timer) oder Sperrmechanismen aktiviert?
- Können Diagnoseberechtigungen durch Session-Wechsel oder ECU-Resets umgangen werden?
- Werden sicherheitsrelevante Ereignisse protokolliert und nachvollziehbar dokumentiert?

Insbesondere in älteren Embedded-Systemen treten häufig Schwachstellen auf, die beispielsweise durch deterministische Seeds, proprietäre und leicht rekonstruierbare Algorithmen oder fehlende Begrenzungen für wiederholte Authentifizierungsversuche verursacht werden.

Vom Funktionstest zur Sicherheitsanalyse

Ein professioneller UDS-Test beschränkt sich deshalb nicht auf die Überprüfung einzelner Diagnosefunktionen. Vielmehr zielt er darauf ab, den gesamten Authentifizierungsprozess zu analysieren und potenzielle Angriffspfade frühzeitig zu erkennen.

Hierzu werden verschiedene Angriffsszenarien simuliert. Ein Beispiel hierfür ist:

- wiederholte Authentifizierungsversuche zur Bewertung des Brute-Force-Schutzes
- Analyse des Seed-Key-Algorithmus auf Vorhersagbarkeit oder kryptografische Schwächen
- Überprüfung von Session-Übergängen und Berechtigungswechseln
- Validierung von Firmware-Download-Mechanismen sowie deren Zugriffskontrollen

Anschließend fließen die gewonnenen Erkenntnisse direkt in die Risikobewertung des Fahrzeugs ein. Sie bilden die Grundlage für weiterführende Analysen, beispielsweise im Rahmen des Firmware-Reviews oder der Secure-Boot-Implementierung. Es geht nicht nur darum, ob ein Security-Mechanismus vorhanden ist, sondern auch darum, wie robust und widerstandsfähig er gegenüber realistischen Angriffen tatsächlich ist.

Root Cause Analysis – Sicherheit beginnt beim Design

Die meisten Schwachstellen im Diagnosebereich entstehen nicht durch einzelne Programmierfehler, sondern bereits während der Entwicklung der Sicherheitsarchitektur. Ein vorhersehbarer Seed, ein veralteter Authentifizierungsalgorithmus oder fehlende Schutzmechanismen gegen wiederholte Zugriffsversuche sind oft die Folge grundlegender Designentscheidungen.

Genau deshalb betrachtet ein White-Box-Assessment nicht nur das Verhalten eines Steuergeräts, sondern analysiert die Ursache einer Schwachstelle. So lassen sich nachhaltige Verbesserungen ableiten und zukünftige Sicherheitsrisiken bereits während der Entwicklung vermeiden.

Diagnosefunktionen sind für die Entwicklung und Wartung unverzichtbar – gleichzeitig gehören sie zu den sensibelsten Schnittstellen eines Fahrzeugs. Ein professionelles Security Assessment bewertet deshalb nicht nur, ob der Zugriff geschützt ist, sondern auch, ob die gesamte Sicherheitsarchitektur unter realistischen Angriffsbedingungen standhält.

Phase 7 – Firmware Review: Vertrauen ist gut – verifizierte Firmware ist besser

Nachdem die Kommunikationsschnittstellen und Diagnosefunktionen analysiert wurden, richtet sich der Fokus auf das Herzstück jedes Steuergeräts: **die Firmware**.

Sie enthält die eigentliche Logik einer ECU, d. h. sie umfasst die Verarbeitung sicherheitskritischer Fahrzeugfunktionen, kryptografische Verfahren sowie Diagnose- und Kommunikationsmechanismen. Fehler oder Schwachstellen in der Firmware können daher weitreichende Auswirkungen auf die Sicherheit des gesamten Fahrzeugs haben. Im Rahmen eines White-Box-Security-Assessments wird die Firmware deshalb als vollständiges Abbild der Sicherheitsarchitektur eines Embedded Systems betrachtet und nicht nur als Software.

Firmware verstehen, bevor sie bewertet wird

Zunächst wird das Firmware-Image extrahiert und dessen Aufbau systematisch analysiert. Dies erfolgt abhängig von der Fahrzeugplattform beispielsweise über bereitgestellte Entwicklungsimagen, Flash-Dumps oder Firmware-Dateien aus dem OTA-Update-Prozess. Das Ziel besteht darin, die interne Struktur der Software sichtbar zu machen und sicherheitsrelevante Komponenten gezielt zu untersuchen.

Für die Analyse werden unter anderem die etablierten Werkzeuge Binwalk und das Firmware-Mod-Kit eingesetzt. Damit ist die Zerlegung von Firmware-Images, die Identifikation eingebetteter Dateisysteme sowie die Untersuchung von Programmcode und Konfigurationsdateien möglich. Der eigentliche Mehrwert entsteht jedoch nicht durch die Extraktion selbst, sondern durch die sicherheitsorientierte Bewertung der extrahierten Inhalte.

Sicherheitsrelevante Artefakte identifizieren

Im Rahmen des Firmware-Reviews werden gezielt Informationen gesucht, die es einem Angreifer ermöglichen würden, unautorisiert auf das System zuzugreifen, oder die Rückschlüsse auf die Sicherheitsarchitektur zulassen würden. Besonders relevant sind dabei unter anderem:

- fest hinterlegte Zugangsdaten (Hardcoded Credentials)
- kryptografische Schlüssel und Zertifikate
- Konfigurationsdateien mit sicherheitsrelevanten Parametern
- aktivierte Debug- oder Service-Schnittstellen
- Bootloader-Komponenten und deren Sicherheitsmechanismen

Auch scheinbar harmlose Konfigurationsdateien können ein erhebliches Risiko darstellen. Ein typisches Beispiel hierfür ist ein im Dateisystem hinterlegter Standardzugang.

```
/etc/config/password.txt
```

```
admin:admin
```

Aus Sicht eines Security Engineers stellt dies nicht nur ein schwaches Passwort dar, sondern einen grundlegenden Verstoß gegen Security-by-Design-Prinzipien. Ein Angreifer könnte diese Zugangsdaten nutzen, um privilegierte Funktionen aufzurufen oder sich dauerhaft Zugang zum Steuergerät zu verschaffen.

Risikobewertung

Critical

Nicht aufgrund des Passworts selbst, sondern weil dadurch die Vertrauenswürdigkeit des gesamten Systems infrage gestellt wird.

Secure Boot – Die Vertrauenskette beginnt beim Start

Neben der eigentlichen Firmware ist die Analyse des Bootprozesses der wichtigste Bestandteil dieser Phase. Ein modernes Steuergerät sollte ausschließlich Firmware ausführen, deren Integrität und Authentizität eindeutig überprüft wurden.

Im Rahmen des Assessments wird deshalb untersucht:

- ob Firmware kryptografisch signiert wird,
- ob digitale Signaturen beim Start tatsächlich validiert werden,
- welche Root-of-Trust-Mechanismen implementiert sind,
- und ob ältere, möglicherweise verwundbare Firmware-Versionen erneut installiert werden können (*Rollback- oder Downgrade-Angriffe*).

Wenn eine dieser Schutzmaßnahmen fehlt oder unzureichend implementiert ist, kann ein Angreifer manipulierte Firmware einschleusen oder bekannte Sicherheitslücken durch das Einspielen älterer Softwareversionen erneut ausnutzen.

Dies kann gerade im Automotive-Umfeld gravierende Folgen haben, da kompromittierte Firmware häufig tief in sicherheitskritische Fahrzeugfunktionen eingreift und dauerhaft auf einem Steuergerät verbleibt.

Firmware Review als Root-Cause-Analyse.

Ein professionelles Firmware-Review endet nicht mit der Identifikation einzelner Schwachstellen. Viel wichtiger ist die Frage, warum sicherheitskritische Informationen überhaupt Bestandteil einer produktiven Firmware sind.

- Warum befinden sich Debug-Schnittstellen noch im Serienimage?
- Weshalb werden Standardzugangsdaten ausgeliefert?
- Warum wird die Integrität der Firmware nicht vollständig geprüft?
- Welche Entwicklungs- oder Build-Prozesse haben diese Schwachstellen ermöglicht?

Diese Fragestellungen führen direkt zur Root-Cause-Analyse.

Denn häufig liegt die Ursache nicht in einem einzelnen Codefragment, sondern in unzureichenden Entwicklungsprozessen, fehlenden Security-Reviews oder einer nicht konsequent umgesetzten Secure-Development-Strategie.

Ein Firmware-Review liefert daher nicht nur technische Ergebnisse, sondern auch wertvolle Erkenntnisse darüber, wie sich Sicherheitsmechanismen bereits während der Entwicklung nachhaltig verbessern lassen.

Firmware ist weit mehr als nur ausführbarer Code: Sie bildet die Vertrauensbasis eines Steuergeräts. Wer ihre Integrität nicht konsequent schützt, riskiert, dass ein Fahrzeug bereits beim Start kompromittiert wird. Effektive Automotive-Cybersecurity beginnt deshalb mit einer überprüfbaren Vertrauenskette vom Bootloader bis zur laufenden Anwendung.

Phase 8 – Python-gestützte Testautomatisierung: Von manuellen Tests zu reproduzierbaren Security Assessments

Ein erfolgreiches Security Assessment besteht nicht nur darin, Schwachstellen zu identifizieren. Genauso entscheidend sind die Fähigkeit, Tests reproduzierbar durchzuführen, Ergebnisse konsistent auszuwerten und Sicherheitsprüfungen effizient in Entwicklungs- und Testprozesse zu integrieren. Genau hier setzt die Testautomatisierung an.

Einzelne Penetrationstests können zwar wertvolle Erkenntnisse liefern, doch erst eine automatisierte Testumgebung ermöglicht es, Sicherheitsprüfungen beliebig oft zu wiederholen, Regressionen frühzeitig zu erkennen und neue Softwarestände systematisch zu validieren.

Dieser Ansatz gewinnt im Automotive-Umfeld zunehmend an Bedeutung. Fahrzeuge bestehen aus einer Vielzahl vernetzter Steuergeräte, deren Software kontinuierlich weiterentwickelt wird. Jede Änderung an einer ECU, einem Diagnoseprozess oder einer Kommunikationsschnittstelle kann unbeabsichtigt neue Sicherheitsrisiken verursachen. Ein modernes White-Box-Security-Assessment zielt deshalb darauf ab, Sicherheit nicht nur einmalig zu prüfen, sondern sie dauerhaft messbar zu machen.

Ein modulares Security-Testframework

Im Rahmen des Assessments wird ein modular aufgebautes Python-Framework entwickelt. Dieses führt unterschiedliche Testbereiche in einer gemeinsamen Architektur zusammen.

Der Aufbau könnte beispielsweise wie folgt aussehen:

```
Security_Test_Framework
├── CAN Tests
├── UDS Tests
├── Automotive Ethernet Tests
├── Firmware Validation
├── Logging
└── Report Generator
```

Jedes Modul übernimmt eine klar definierte Aufgabe und kann unabhängig davon erweitert oder angepasst werden. So entsteht eine flexible Testplattform, die sich sowohl für einzelne Komponenten als auch für vollständige End-to-End-Assessments eignet. Dieser modulare Ansatz erleichtert die Wartung des Frameworks und unterstützt die Zusammenarbeit zwischen Security Engineers, Testteams und Entwicklern.

Automatisierte Diagnoseprüfungen

Ein Beispiel hierfür ist die automatisierte Validierung des „UDS Security Access“. Anstatt jede Diagnoseanfrage manuell zu senden und auszuwerten, kann ein Testskript, das mit Python erstellt wurde, den gesamten Ablauf selbstständig durchführen und dokumentieren.

```
class UDSTest:
    def security_access_test(self):
        response = send_request([0x27, 0x01])
        if response:
            print("Security Access available")
```

In einer produktiven Testumgebung wäre ein solcher Test selbstverständlich deutlich umfangreicher. Neben der reinen Verfügbarkeit des Dienstes könnten beispielsweise die folgenden Aspekte automatisiert geprüft werden:

- korrekte Antworten auf gültige und ungültige Requests,
- Verhalten nach wiederholten Fehlversuchen,
- Aktivierung von Delay-Mechanismen,
- Reaktion auf fehlerhafte Session-Wechsel,

- vollständige Protokollierung aller Testergebnisse.

Dadurch entstehen reproduzierbare Testfälle, die unabhängig vom jeweiligen Tester jederzeit erneut ausgeführt werden können, was die Wiederholbarkeit und Reproduzierbarkeit der Testfälle sicherstellt.

Automatisierung als Grundlage kontinuierlicher Sicherheit

Die eigentliche Stärke automatisierter Security-Tests liegt jedoch nicht in der Zeitersparnis. Sie ermöglichen vielmehr eine kontinuierliche Überprüfung der Sicherheit.

Jede neue Firmware-Version, jede Anpassung an einer ECU sowie jede Änderung innerhalb der Fahrzeugarchitektur kann automatisch gegen dieselben Sicherheitsanforderungen validiert werden. Dadurch lassen sich Regressionen frühzeitig erkennen und Sicherheitsmechanismen dauerhaft überwachen.

Insbesondere im Zusammenwirken mit Continuous Integration (CI) und Continuous Testing ergeben sich dadurch Entwicklungsprozesse, in denen die Sicherheit nicht erst kurz vor der Serienfreigabe berücksichtigt wird, sondern integraler Bestandteil jeder Softwareänderung ist.

Automatische Auswertung und Reporting

Ebenso wichtig wie die Durchführung der Tests ist die strukturierte Aufbereitung der Ergebnisse. Ein professionelles Security Assessment endet nicht mit einer Konsolenausgabe, sondern mit nachvollziehbaren Berichten, die von Security Engineers und Entwicklungsteams weiterverarbeitet werden können.

Deshalb erzeugt das Framework automatisiert Berichte in standardisierten Formaten wie:

- **JSON** zur maschinellen Weiterverarbeitung,
- **XML** für bestehende Test- und Qualitätssicherungssysteme,
- **HTML** für übersichtliche Management- und Entwicklerberichte.

Mithilfe dieser standardisierten Dokumentation sind die Ergebnisse und Findings nachvollziehbar, reproduzierbar und langfristig vergleichbar.

Testautomatisierung als Bestandteil der Root Cause Analysis

Automatisierung dient nicht nur der Effizienz. Sie ermöglicht es, identifizierte Schwachstellen gezielt nachzustellen und nach einer Korrektur erneut zu überprüfen. So lässt sich eindeutig nachvollziehen, ob eine Sicherheitsmaßnahme die Ursache eines Problems tatsächlich beseitigt oder nur dessen Symptome verdeckt.

Automatisierte Tests werden so zu einem zentralen Werkzeug der Root-Cause-Analyse. Sie liefern unabhängig von der Häufigkeit von Veränderungen an Software oder Fahrzeugarchitektur objektive und wiederholbare Nachweise darüber, ob Sicherheitsmechanismen dauerhaft wirksam sind

Ein einzelner erfolgreicher Penetrationstest beweist lediglich, dass eine Schwachstelle existiert. Ein automatisiertes Security-Framework stellt sicher, dass dieselbe Schwachstelle niemals unbemerkt zurückkehrt. Genau darin liegt der Unterschied zwischen einer punktuellen Sicherheitsprüfung und nachhaltigem Security Engineering.

Phase 9 – Risikobewertung: Aus Schwachstellen werden fundierte Entscheidungen

Die Identifikation einer Sicherheitslücke ist der Ausgangspunkt für eine fundierte Risikobewertung. Ein Security Engineer muss technische Findings in einen nachvollziehbaren Risikokontext einordnen und ihre Relevanz bewerten. Dieser Schritt entscheidet, welche Maßnahmen vor der Serienfreigabe umgesetzt werden müssen.

Risiken systematisch bewerten

Im Rahmen des Assessments werden alle Ergebnisse anhand objektiver Bewertungskriterien analysiert. Dabei stehen insbesondere die folgenden drei Fragestellungen im Mittelpunkt:

- **Impact:** Welche Auswirkungen hätte eine erfolgreiche Ausnutzung der Schwachstelle auf Fahrzeugfunktionen, Datenschutz oder funktionale Sicherheit?
- **Exploitability:** Wie realistisch und mit welchem technischen Aufwand lässt sich die Schwachstelle tatsächlich ausnutzen?
- **Eintrittswahrscheinlichkeit:** Unter welchen Bedingungen kann der Angriff erfolgen und wie wahrscheinlich ist dieses Szenario im realen Betrieb?

Durch die Kombination dieser Faktoren ist eine nachvollziehbare Priorisierung möglich. Dadurch werden Entwickler und Projektverantwortliche bei der gezielten Umsetzung von Sicherheitsmaßnahmen unterstützt.

Beispiel einer Risikobewertung

Ein typisches Ergebnis könnte ein unzureichend geschützter UDS-Security-Access sein. Wenn der Authentifizierungsmechanismus nicht ausreichend abgesichert ist oder Schutzmaßnahmen gegen unautorisierte Zugriffe fehlen, besteht das Risiko, dass privilegierte Diagnosefunktionen missbraucht werden können.

Finding: UDS Security Access ohne ausreichende Zugriffsschutzmechanismen.

CVSS-Bewertung Score: 8.6 – High. Entscheidend hier ist die technische Begründung für das Risikoniveau.

Die Bewertung ergibt sich aus mehreren sicherheitsrelevanten Faktoren:

- Die Schwachstelle könnte über erreichbare Kommunikationsschnittstellen ausgenutzt werden
- Ein erfolgreicher Angriff ermöglicht den Zugriff auf privilegierte ECU-Funktionen
- Sicherheitskritische Fahrzeugfunktionen könnten beeinflusst oder manipuliert werden
- Durch den möglichen Einfluss auf Steuergeräte besteht ein potenzieller Bezug zur funktionalen Sicherheit

Risiken im Kontext der Fahrzeugarchitektur betrachten

Ein ungeschützter Diagnosezugang ist beispielsweise relevanter, wenn er ausschließlich lokal über die OBD-II-Schnittstelle erreichbar ist, als wenn dieselbe Funktion über eine Telematik-Einheit oder eine externe Netzwerkverbindung indirekt angesprochen werden kann. Deshalb werden Befunde stets im Zusammenhang mit der zuvor analysierten Fahrzeugarchitektur, den Kommunikationspfaden und den identifizierten Angriffsszenarien bewertet. Erst dieser Gesamtkontext ermöglicht eine realistische Einschätzung des tatsächlichen Risikos.

Risikobewertung als Grundlage für technische Entscheidungen

Eine gute Risikobewertung beantwortet nicht nur die Frage, wie kritisch eine Schwachstelle ist, sondern auch, mit welchen Maßnahmen sich der größte Sicherheitsgewinn erzielen lässt. Dadurch erhalten Entwicklungsteams eine Priorisierung:

- Welche Schwachstellen müssen zwingend vor der Serienfreigabe behoben werden?

- Welche Risiken können durch zusätzliche Sicherheitsmechanismen reduziert werden?
- Welche Maßnahmen haben den größten Einfluss auf die Gesamtsicherheit des Fahrzeugs?

Diese Priorisierung schafft mehr Transparenz und hilft Projektteams, Entwicklungsaufwand und Sicherheitsanforderungen sinnvoll miteinander zu verbinden.

Risikobewertung als Bestandteil der Root Cause Analysis

Eine hohe Einstufung allein löst kein Sicherheitsproblem. Deshalb endet die Auswertung nicht beim CVSS-Score.

Für jedes kritische Ergebnis wird untersucht,

- warum die Schwachstelle entstanden ist,
- welche Architektur- oder Implementierungsentscheidung dazu geführt hat,
- und welche nachhaltigen Maßnahmen erforderlich sind, um vergleichbare Risiken künftig zu vermeiden.

Die Risikobewertung wird somit zu einem wesentlichen Bestandteil der Root-Cause-Analyse und bildet die Grundlage für Security-by-Design-Entscheidungen in zukünftigen Entwicklungsprojekten.

Die Aufgabe eines Security Engineers besteht nicht darin, möglichst viele Schwachstellen zu finden. Vielmehr ist es entscheidend, ihre Auswirkungen auf das Gesamtsystem zu verstehen, Risiken nachvollziehbar zu priorisieren und daraus konkrete technische Entscheidungen abzuleiten. So wird aus einem Finding eine fundierte Sicherheitsstrategie.

Phase 10 – Management Summary: Komplexe Sicherheitsrisiken verständlich kommunizieren

Ein erfolgreiches Security Assessment liefert mehr als eine Liste technischer Schwachstellen. Seine Erkenntnisse müssen in Entscheidungen für Entwicklung, Projektmanagement und Unternehmensführung übersetzt werden. Die Management Summary verbindet technische Analyse und strategische Entscheidung. Die Management Summary beantwortet eine andere Frage: Welches Risiko besteht für das Fahrzeug und welche Maßnahmen sollten vor der Serienfreigabe umgesetzt werden? Es geht um eine klare und nachvollziehbare Darstellung der wesentlichen Erkenntnisse, nicht um technische Details. Das Ziel ist, dass auch Personen ohne tiefgehendes Cybersecurity-Wissen Risiken richtig einordnen und fundierte Entscheidungen treffen können.

Executive Summary der Sicherheitsbewertung

Im Rahmen der End-to-End-Security-Validation wurden die Architektur, die Kommunikationsschnittstellen, die Diagnosefunktionen, die Firmware sowie die sicherheitsrelevanten Implementierungen systematisch analysiert. Die Bewertung zeigt, dass die grundlegende Fahrzeugarchitektur bereits zahlreiche Sicherheitsmechanismen integriert. Gleichzeitig wurden mehrere Schwachstellen identifiziert, die vor einer Serienfreigabe behoben werden sollten, um potenzielle Angriffsrisiken nachhaltig zu reduzieren.

Besonders relevant sind dabei folgende Themen:

- Unzureichend abgesicherte Diagnosefunktionen: Unter bestimmten Voraussetzungen könnten sie den Zugriff auf privilegierte ECU-Funktionen ermöglichen.
- Fehlende oder unvollständige Firmware-Verifikation, wodurch die Integrität installierter Software nicht in allen Szenarien zuverlässig sichergestellt werden kann.
- Nicht authentifizierte CAN-Kommunikation: Dadurch sind manipulierte Nachrichten innerhalb des Fahrzeugnetzwerks grundsätzlich möglich, sofern ein Angreifer Zugriff auf den Bus erhält.

Diese Schwachstellen betreffen sicherheitskritische Komponenten der Fahrzeugarchitektur und können Auswirkungen auf Integrität, Verfügbarkeit und funktionale Sicherheit haben.

Risiken priorisieren statt nur dokumentieren

Ein zentrales Ergebnis des Assessments ist die Priorisierung der identifizierten Risiken. Nicht jede Schwachstelle hat die gleiche Relevanz für die Serienfreigabe. Deshalb wurden die Findings bewertet. Es zeigt sich, dass insbesondere jene Schwachstellen priorisiert behandelt werden sollten, die

- den unautorisierten Zugriff auf Steuergeräte ermöglichen,
- die Integrität von Firmware oder Software-Updates gefährden,
- sicherheitskritische Kommunikationspfade zwischen den ECUs betreffen.

Diese Priorisierung unterstützt Projektverantwortliche dabei, Entwicklungsressourcen gezielt einzusetzen und Sicherheitsmaßnahmen dort umzusetzen, wo sie den größten Einfluss auf das Gesamtrisiko haben.

Empfehlungen vor der Serienfreigabe

Die technischen Analysen ergeben mehrere konkrete Handlungsempfehlungen, die vor der Freigabe des Fahrzeugs umgesetzt werden müssen. Dazu gehören unter anderem folgende Punkte:

- Härtung der Diagnosezugriffe durch robuste Authentifizierungs- und Autorisierungsmechanismen
- Vollständige Implementierung einer kryptografisch abgesicherten Firmware-Verifikation einschließlich Secure Boot und Schutz vor Downgrade-Angriffen
- Zusätzliche Sicherheitsmechanismen zum Schutz der internen Fahrzeugkommunikation, beispielsweise durch Authentifizierung sicherheitskritischer Nachrichten
- Integration automatisierter Security-Tests in den Entwicklungsprozess, um zukünftige Softwareänderungen kontinuierlich gegen definierte Sicherheitsanforderungen zu validieren

Diese Maßnahmen tragen dazu bei, identifizierte Risiken nachhaltig zu reduzieren und die langfristige Cyber-Resilienz des Fahrzeugs gleichzeitig zu stärken.

Sicherheit als kontinuierlicher Entwicklungsprozess

Die Ergebnisse des Assessments zeigen deutlich, dass moderne Fahrzeugsicherheit nicht durch eine einzelne Schutzmaßnahme erreicht wird. Von entscheidender Bedeutung ist das Zusammenspiel aus sicherer Architektur, robuster Softwareentwicklung, kontinuierlicher Sicherheitsvalidierung und konsequenter Risikobewertung. Eine Management Summary dient somit nicht nur der Berichterstattung. Sie schafft Transparenz, unterstützt fundierte Entscheidungen und bildet die Grundlage für die Priorisierung weiterer Entwicklungsmaßnahmen

Gerade im Automotive-Umfeld, in dem Sicherheit, Qualität und Time-to-Market gleichermaßen berücksichtigt werden müssen, ist diese Form der Kommunikation ein wesentlicher Bestandteil eines erfolgreichen Security-Engineering-Prozesses.

Die wichtigste Erkenntnis eines Security Assessments ist nicht die Anzahl der gefundenen Schwachstellen. Entscheidend ist, welche Risiken tatsächlich relevant sind, welche Auswirkungen sie auf das Fahrzeug haben und mit welchen Maßnahmen sich eine sichere Serienfreigabe am besten erreichen lässt. Genau diese Transparenz schafft eine professionelle Management Summary.

Phase 11 – Technischer Abschlussbericht: Aus Findings werden umsetzbare Sicherheitsmaßnahmen

Ein erfolgreiches Security Assessment zeichnet sich durch eine strukturierte Dokumentation aller Ergebnisse aus. Diese ermöglicht es Entwicklungsteams, identifizierte Schwachstellen nachzuvollziehen, Ursachen zu verstehen und geeignete Gegenmaßnahmen umzusetzen. Der technische Abschlussbericht ist somit das zentrale Ergebnis eines White-Box-Security-Assessments. Er dokumentiert nicht nur, welche Schwachstellen identifiziert wurden, sondern beschreibt auch nachvollziehbar, wie sie entstanden sind, welche Auswirkungen sie haben und welche Maßnahmen zu ihrer nachhaltigen Behebung empfohlen werden. Damit wird der Bericht zu einem verbindlichen Referenzdokument für die Bereiche Entwicklung, Qualitätssicherung, Projektmanagement und zukünftige Sicherheitsbewertungen.

Transparenz und Nachvollziehbarkeit als oberstes Ziel

Ein professioneller Abschlussbericht hat das Ziel, sämtliche Ergebnisse reproduzierbar und technisch nachvollziehbar zu dokumentieren. Jedes Finding muss so beschrieben werden, dass ein Entwickler die Schwachstelle reproduzieren, ihre Ursache verstehen und die vorgeschlagenen Maßnahmen gezielt umsetzen kann. Gleichzeitig dient der Bericht als Nachweis gegenüber internen Security-Abteilungen, Auditoren oder OEMs, dass die Sicherheitsvalidierung systematisch und nach definierten Methoden durchgeführt wurde.

Struktur eines technischen Security Reports

Um eine einheitliche und nachvollziehbare Dokumentation sicherzustellen, folgt der Abschlussbericht einer klar definierten Struktur.

- **Executive Summary:** Eine kompakte Zusammenfassung der wichtigsten Ergebnisse und identifizierten Risiken für die Projektleitung und das Management.
- **Assessment Scope:** Beschreibung der untersuchten Steuergeräte, Kommunikationsschnittstellen und Firmware-Komponenten sowie der definierten Testgrenzen.
- **Systemarchitektur:** Darstellung der Fahrzeugarchitektur, der Kommunikationspfade, der Trust Boundaries und der sicherheitsrelevanten Komponenten.
- **Testmethodik:** Dokumentation der eingesetzten Verfahren wie White-Box-Code-Reviews, Threat Modeling, Kommunikationsanalysen, UDS-Tests, Firmware-Reviews sowie automatisierte, Python-basierte Security-Tests.
- **Technische Findings:** Detaillierte Beschreibung sämtlicher identifizierter Schwachstellen einschließlich Risikobewertung, technischer Analyse und konkreter Handlungsempfehlungen.

Diese Struktur stellt sicher, dass alle Beteiligten – vom Entwickler bis zum Projektmanager – dieselbe Informationsbasis nutzen und Sicherheitsmaßnahmen nachvollziehbar priorisieren können.

Ein Beispiel für ein technisches Finding: Ein professionelles Finding besteht nicht nur aus einer Problembeschreibung. Es dokumentiert vielmehr den gesamten Lebenszyklus einer Schwachstelle – von der technischen Beobachtung bis hin zur empfohlenen Gegenmaßnahme.

Finding SEC-001

- **Titel:** Unzureichend abgesicherter UDS Security Access
- **Beschreibung:** Während der Analyse der Diagnosefunktionen wurde festgestellt, dass der Security-Access-Mechanismus wiederholte Authentifizierungsanfragen ohne wirksame Schutzmaßnahmen verarbeitet. Dadurch kann ein Angreifer den Authentifizierungsprozess automatisiert analysieren und unter bestimmten Voraussetzungen Angriffe gegen den Seed-Key-Mechanismus durchführen.
- **Technische Auswirkung:** Ein erfolgreicher Angriff könnte den unautorisierten Zugriff auf privilegierte Diagnosefunktionen ermöglichen und damit sicherheitsrelevante ECU-Funktionen freischalten.
- **Risikobewertung:** High - Die Bewertung ergibt sich aus der Kombination einer erreichbaren Diagnosefunktion, fehlender Schutzmechanismen gegen wiederholte Zugriffsversuche sowie der möglichen Beeinflussung sicherheitskritischer Steuergeräte.

Empfohlene Maßnahmen

- Implementierung eines effektiven Rate Limitings zur Begrenzung wiederholter Authentifizierungsversuche.
- Einsatz robuster kryptografischer Verfahren für den Seed-Key-Mechanismus.
- Einführung sicherer Authentifizierungs- und Autorisierungskonzepte.
- Vollständige Protokollierung sicherheitsrelevanter Diagnoseereignisse zur Unterstützung von Monitoring und Incident Response.

Dieses Beispiel zeigt, dass ein Finding weit mehr als nur eine Schwachstellenbeschreibung umfasst. Es verbindet technische Analyse, Risikobewertung und konkrete Handlungsempfehlungen zu einer nachvollziehbaren Entscheidungsgrundlage.

Dokumentation als Bestandteil der Root Cause Analysis

Ein professioneller Abschlussbericht hat nicht nur die Aufgabe, zu dokumentieren, was gefunden wurde, sondern auch, warum eine Schwachstelle entstanden ist. Deshalb enthält jedes Finding zusätzlich eine Ursachenanalyse.

In dem beschriebenen Beispiel liegt die Ursache nicht im wiederholten Diagnosezugriff selbst, sondern in der unzureichenden Sicherheitsarchitektur des Authentifizierungsmechanismus.

Erst durch diese Analyse sind Entwicklungsteams in der Lage, nicht nur die konkrete Schwachstelle zu beheben, sondern auch vergleichbare Sicherheitsprobleme in zukünftigen Softwareversionen von vornherein zu vermeiden. Der Abschlussbericht wird so zu einem wichtigen Instrument der kontinuierlichen Verbesserung im Secure Development Lifecycle.

Mehr als ein Bericht – eine Grundlage für sichere Entwicklung

Ein technischer Abschlussbericht schafft Transparenz über den Sicherheitszustand eines Fahrzeugs, hilft dabei, technische Maßnahmen zu priorisieren, und bildet die Grundlage für zukünftige Security Reviews, Regressionstests und Serienfreigaben. Eine nachvollziehbare Dokumentation fördert darüber hinaus die Zusammenarbeit zwischen Security Engineers, Softwareentwicklern, Systemarchitekten und Qualitätssicherung. Alle Beteiligten arbeiten auf Basis derselben technischen Erkenntnisse und können Maßnahmen somit gezielt planen und umsetzen.

Ein professioneller Security-Report beschreibt nicht nur eine Schwachstelle, sondern auch ihre gesamte Entstehungsgeschichte. Von der Ursache über die technische Analyse bis hin zur nachhaltigen Lösung. Dadurch wird aus einer Schwachstelle eine konkrete Verbesserung für das gesamte Fahrzeug.

Phase 12 – Empfehlungen für die Entwicklung: Von Security Testing zu Security Engineering

Ein Security Assessment entfaltet seinen größten Mehrwert durch die Anzahl identifizierter Schwachstellen und die Qualität der abgeleiteten Verbesserungsmaßnahmen. Die Aufgabe eines Automotive Security Engineers endet daher nicht mit dem technischen Abschlussbericht. Sie beginnt vielmehr dort, wo Erkenntnisse in konkrete Entwicklungsmaßnahmen überführt werden. In dieser Phase geht es nicht nur darum, die identifizierten Risiken zu beheben, sondern auch darum, ihre Ursachen dauerhaft aus der Software- und Systemarchitektur zu entfernen. Der Fokus liegt dabei nicht auf der kurzfristigen Korrektur einzelner Ergebnisse, sondern auf dem Aufbau einer Sicherheitsarchitektur, die zukünftigen Bedrohungen bereits auf Design- und Implementierungsebene begegnet.

Die zentralen technischen Empfehlungen, die sich aus dem White-Box-Security-Assessment ableiten lassen, werden im Folgenden zusammengefasst.

1. Kommunikation absichern – Vertrauen in Fahrzeugnetzwerke schaffen

Die Analyse der Fahrzeugkommunikation hat gezeigt, dass klassische Bussysteme wie CAN nicht für moderne Cyberbedrohungen ausgelegt sind. Nachrichten werden standardmäßig weder authentifiziert noch verschlüsselt. Dadurch besteht die Möglichkeit, dass ein Angreifer, der Zugriff auf den Bus erhält, manipulierte Frames in das Netzwerk einschleust. Es wird nicht empfohlen, den CAN-Bus vollständig zu ersetzen, sondern seine Sicherheitsmechanismen gezielt zu erweitern. Eine empfohlene Sicherheitsmaßnahme ist die Einführung von Secure Onboard Communication (SecOC). Zusätzlich wird die Verwendung von Message Authentication Codes (MAC) empfohlen.

SecOC: Ein zentraler Baustein moderner Fahrzeugarchitekturen ist die Secure Onboard Communication (SecOC) nach AUTOSAR-Standard. SecOC schützt sicherheitskritische Nachrichten durch kryptografische Authentifizierung und gewährleistet, dass empfangene Daten tatsächlich von einer vertrauenswürdigen ECU stammen und während der Übertragung nicht verändert wurden.

Dadurch lassen sich unter anderem folgende Angriffe deutlich erschweren:

- CAN Message Injection
- Replay-Angriffe
- Manipulation sicherheitskritischer Steuerbefehle
- Identitätsfälschung einzelner ECUs

MAC: Einsatz kryptografischer Message Authentication Codes (MAC). Hierbei wird jede sicherheitskritische Nachricht mit einem kryptografischen Prüfwert versehen, der vom empfangenden Steuergerät validiert wird.

Dadurch kann jede ECU unmittelbar erkennen,

- ob eine Nachricht manipuliert wurde,
- ob sie tatsächlich vom erwarteten Kommunikationspartner stammt,
- oder ob ein Replay-Angriff vorliegt.

Insbesondere in Kombination mit SecOC ergibt sich daraus eine robuste Vertrauenskette innerhalb der Fahrzeugkommunikation.

2. Firmware absichern – Vertrauen beginnt bereits beim Start

Secure-Boot: Die Firmware eines Steuergeräts bildet die Vertrauensbasis sämtlicher Fahrzeugfunktionen. Deshalb sollte jede ECU ausschließlich Software ausführen, deren Herkunft und Integrität eindeutig nachgewiesen werden können. Dies kann durch Secure Boot konsequent umgesetzt werden. Vor dem Start einer ECU muss überprüft werden, ob die Firmware unverändert ist und vom Hersteller freigegeben wurde.

Ein vollständiger Secure-Boot-Prozess sollte daher

- eine unveränderbare Root of Trust besitzen,
- digitale Signaturen kryptografisch validieren,
- manipulierte Firmware konsequent ablehnen,
- und sämtliche Prüfungen revisionssicher protokollieren.

Erst dadurch entsteht eine durchgängige Vertrauenskette vom Bootloader bis zur laufenden Anwendung.

Kryptografisch signierte Firmware: Jedes Firmware-Image sollte digital signiert werden. Die ECU darf Firmware nur akzeptieren, wenn Signatur, Zertifikatskette und Integrität erfolgreich validiert wurden. So wird verhindert, dass manipulierte oder nicht autorisierte Software auf einem Steuergerät ausgeführt werden kann.

Schutz vor Rollback- und Downgrade-Angriffen: Eine häufig unterschätzte Schwachstelle ist die Möglichkeit, ältere Firmware-Versionen erneut zu installieren. Auch wenn aktuelle Software bereits bekannte Sicherheitslücken behebt, können Angreifer gezielt auf frühere Versionen zurückwechseln und diese Schwachstellen erneut ausnutzen. Deshalb sollte jede ECU zusätzlich eine Rollback-Protection implementieren. Dabei wird die installierte Firmware-Version kryptografisch abgesichert, sodass ein Zurückspielen älterer Versionen zuverlässig verhindert wird.

3. Diagnose absichern – Wartung ermöglichen, Missbrauch verhindern

Diagnosefunktionen zählen zu den leistungsfähigsten Schnittstellen eines Fahrzeugs. Sie ermöglichen Softwareupdates, Konfigurationsänderungen und Wartungsarbeiten, stellen jedoch gleichzeitig eine besonders attraktive Angriffsfläche dar.

Starke Authentifizierung: Der Zugriff auf privilegierte Diagnosefunktionen sollte ausschließlich nach einer starken Authentifizierung erfolgen. Der bisherige Seed-Key-Mechanismus sollte dabei kontinuierlich weiterentwickelt und durch aktuelle kryptografische Verfahren ergänzt werden. Zusätzlich sollten

- Challenge-Response-Verfahren,
- Hardware Security Module (HSM),
- sowie zertifikatsbasierte Authentifizierung

in zukünftigen Fahrzeugplattformen berücksichtigt werden.

Granulare Zugriffssteuerung: Nicht alle Diagnosefunktionen benötigen dieselben Berechtigungen. Deshalb sollte der Zugriff konsequent nach Rollen und Berechtigungsstufen getrennt werden. So benötigt ein Diagnosetester für Wartungsarbeiten beispielsweise andere Rechte als ein Produktionssystem oder ein OTA-Update-Service. Durch eine feingranulare Zugriffskontrolle lassen sich unnötige Berechtigungen vermeiden und potenzielle Angriffsflächen deutlich reduzieren.

4. Security-by-Design – Sicherheit beginnt nicht beim Penetrationstest

Die wirkungsvollste Sicherheitsmaßnahme ist es, Schwachstellen gar nicht erst entstehen zu lassen. Deshalb sollte Cybersicherheit bereits fester Bestandteil des Entwicklungsprozesses sein und nicht erst während der abschließenden Sicherheitsvalidierung berücksichtigt werden.

Threat Modeling frühzeitig integrieren: Bereits während der Systementwicklung sollten alle Komponenten hinsichtlich möglicher Angriffsszenarien bewertet werden. Methoden wie

- STRIDE,
- Attack Trees
- oder TARA gemäß ISO/SAE 21434

helfen dabei, Risiken frühzeitig zu identifizieren und Sicherheitsmaßnahmen bereits auf Architekturebene zu berücksichtigen.

Secure Coding Guidelines etablieren: Softwareentwickler sollten verbindliche Richtlinien für sicherheitskritische Programmierung anwenden. Dazu gehören unter anderem

- konsequente Eingabevalidierung,
- sichere Speicherverwaltung,
- Vermeidung unsicherer Standardbibliotheken,
- sichere Nutzung kryptografischer Bibliotheken,
- sowie die konsequente Behandlung sicherheitsrelevanter Fehler.

Gerade im Embedded-Bereich leisten Standards wie MISRA C, CERT C oder AUTOSAR einen wichtigen Beitrag zur Reduzierung typischer Implementierungsfehler.

Security Code Reviews als Standard etablieren: Automatisierte Analysen sind wertvoll – sie ersetzen jedoch keine sicherheitsorientierte Codeprüfung. Deshalb sollte jede sicherheitskritische Änderung zusätzlich durch strukturierte Security Code Reviews begleitet werden. Die Kombination aus

- manueller White-Box-Analyse,
- statischer Codeanalyse (SAST),
- dynamischen Sicherheitstests,
- und automatisierten Regressionstests

ermöglicht eine deutlich frühere Identifikation sicherheitsrelevanter Schwachstellen und führt langfristig zu einer besseren Softwarequalität.

Sicherheit als kontinuierlicher Entwicklungsprozess

Die Ergebnisse des Assessments zeigen eindeutig, dass nachhaltige Cybersicherheit nicht durch einzelne Schutzmechanismen erreicht wird. Erst das Zusammenwirken von sicherer Kommunikation, vertrauenswürdiger Firmware, geschützten Diagnosefunktionen und einem konsequenten Secure-Development-Lifecycle-Prozess führt zu einer widerstandsfähigen Fahrzeugarchitektur. Security sollte daher nicht als zusätzliche Funktion betrachtet werden, sondern als integraler Bestandteil jeder Entwicklungsentscheidung – von der ersten Architekturzeichnung bis zur Serienfreigabe.

Die wirksamste Sicherheitsmaßnahme besteht nicht darin, eine einzelne Schwachstelle zu beheben. Es geht vielmehr darum, Entwicklungsprozesse so zu gestalten, dass diese Schwachstelle künftig gar nicht mehr entstehen kann. Genau darin liegt der Kern der modernen Automotive-Cybersecurity: Security-by-Design statt Security-by-Reaction.